

DSA Quick Cheat Sheet

A short PDF for testing upload, reading, and summarization.

1. Big-O Basics

- **$O(1)$** - constant time, like accessing `arr[i]`
- **$O(\log n)$** - halves the problem each step, like binary search
- **$O(n)$** - linear scan through all items
- **$O(n \log n)$** - common in efficient sorting algorithms
- **$O(n^2)$** - nested loops, often slow for large inputs

Tip: always mention time and space complexity when describing a solution.

2. Core Data Structures

- **Array** - fast random access, slow insert/delete in the middle
- **Linked List** - easy insert/delete, slower access
- **Stack** - LIFO; operations: push, pop, peek
- **Queue** - FIFO; operations: enqueue, dequeue, front
- **HashMap** - fast average lookup by key
- **Tree / BST** - hierarchical structure; BST keeps $\text{left} < \text{root} < \text{right}$
- **Graph** - nodes connected by edges; can be directed or undirected

Tip: choose a structure based on the operation you need most.

3. Must-Know Traversals

- **Binary Tree DFS** - preorder, inorder, postorder
- **Binary Tree BFS** - level order traversal using a queue
- **Graph DFS** - uses stack/recursion; good for components and paths
- **Graph BFS** - finds shortest path in unweighted graphs

Tip: recursion is often used for tree DFS.

4. Sorting Summary

- **Bubble Sort** - $O(n^2)$, simple but slow
- **Selection Sort** - $O(n^2)$, few swaps
- **Insertion Sort** - good for nearly sorted data
- **Merge Sort** - $O(n \log n)$, stable, uses extra space
- **Quick Sort** - average $O(n \log n)$, worst $O(n^2)$
- **Heap Sort** - $O(n \log n)$, in-place

Tip: merge sort and quick sort are the most common exam answers.

5. Common Interview Patterns

- Two pointers

- Sliding window
- Binary search on answer
- Fast and slow pointers
- Prefix sums
- Backtracking
- Dynamic programming

Tip: many problems are just a pattern in disguise.

6. Tiny Practice Examples

- Find maximum value in an array -> $O(n)$
- Search sorted array -> binary search, $O(\log n)$
- Check balanced parentheses -> stack
- Count word frequency -> hash map
- Shortest path in unweighted graph -> BFS

Tip: explain your choice before writing code.